Frontend Assignment: Public API Integration & Beautiful UI

Assignment Title

Build a Beautiful Pokémon Explorer Using a Public API

Objective

Create a modern, responsive frontend application that consumes a public API and presents the data through a polished, user-friendly interface.

You will use the **PokéAPI**, a free public API that requires no authentication.

Your goal is not only to make the API work, but also to demonstrate strong frontend skills through **UI design, API integration, loading states, error handling, responsiveness, and component architecture**.

API

Use:

PokéAPI

Base URL:

`https://pokeapi.co/api/v2/`

Useful endpoints:

- `GET /pokemon?limit=20&offset=0` — Get a list of Pokémon
- `GET /pokemon/{name}` — Get detailed information about a Pokémon
- `GET /pokemon/{id}` — Get Pokémon information by ID
- `GET /type/{type}` — Get Pokémon belonging to a specific type

No API key is required.

Core Requirements

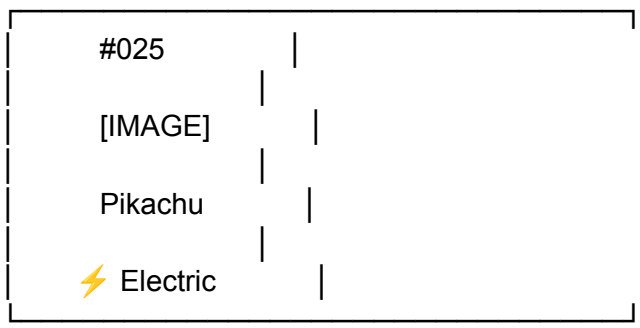
1. Pokémon Listing

Create a homepage that displays Pokémon in a beautiful card-based layout.

Each card should contain:

- Pokémon image
- Pokémon name
- Pokémon ID
- Pokémon type(s)
- Basic visual styling based on the Pokémon's type

Example:



The UI should feel like a real production application rather than a basic API demo.

---

2. Search

Add a search bar that allows users to search for a Pokémon by name.

Example:

Search Pokémon...

When the user searches for `pikachu`, fetch the relevant Pokémon from the API and display its information.

Handle cases where the Pokémon does not exist.

Example error state:

Pokémon not found. Try searching for another Pokémon.

---

3. Pagination / Load More

Do not load every Pokémon at once.

Implement either:

- Pagination
- "Load More" button
- Infinite scrolling

The preferred option is **Load More**.

Example:

[Pokémon Cards]

Load More
↓

---

4. Pokémon Details

When a user clicks a Pokémon card, open a detailed view.

You can implement this as either:

- A separate route/page
- A modal
- A side drawer

The details view should include:

- Large Pokémon image
- Name
- ID
- Types
- Height
- Weight
- Abilities
- Base statistics
- Basic information about its moves

Example statistics:

---

5. Filtering

Add a filter that allows users to filter Pokémon by type.

Example:

All
Fire
Water
Grass
Electric
Psychic
Dragon
Ghost
...

Selecting a type should update the displayed Pokémon.

---

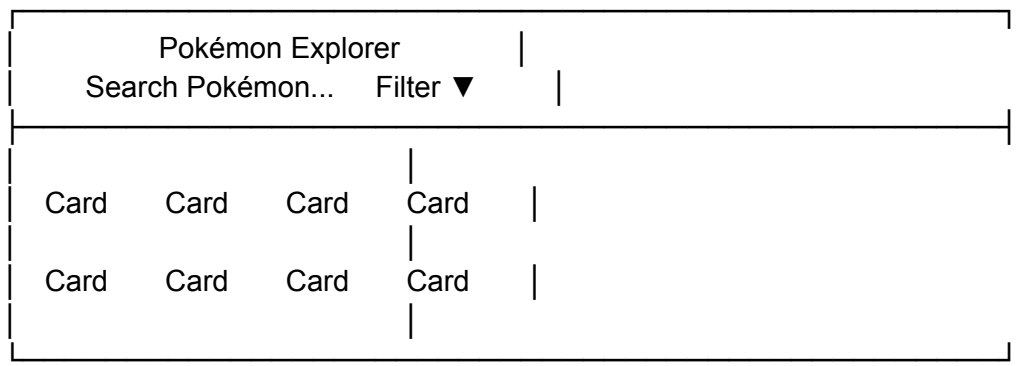
6. Responsive Design

The application must work properly on:

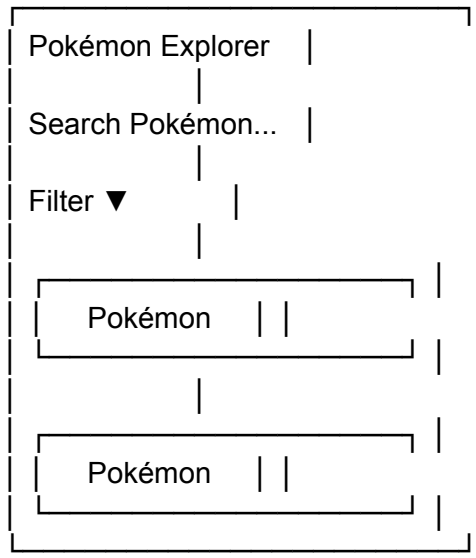
- Desktop
- Tablet
- Mobile

Suggested layout:

Desktop



Mobile



---

7. Loading State

Do not leave the screen blank while waiting for the API.

Create a beautiful loading experience.

For example:

- Skeleton cards
- Animated loader
- Shimmer effect

Avoid simply displaying:

Loading...

A polished skeleton UI is preferred.

---

8. Error Handling

Handle API failures gracefully.

Your application should display a proper error state if:

- API request fails
- Pokémon does not exist
- Network connection fails
- Unexpected API response occurs

Include a **Retry** button where appropriate.

Example:

Something went wrong.

We couldn't load the Pokémon.

[Try Again]

---

9. Empty State

If a search or filter produces no results, show a useful empty state.

Example:

🔍

No Pokémon found.

Try searching for a different Pokémon.

UI/UX Requirements

The visual design is an important part of this assignment.

Your application should include:

Modern Design

Use:

- Clean typography
- Consistent spacing
- Rounded cards
- Subtle shadows
- Smooth hover effects
- Good color contrast
- Clear visual hierarchy

Animations

Add subtle animations such as:

- Card hover animation
- Button hover states
- Modal/drawer transitions
- Skeleton loading animation
- Page transitions where appropriate

Do not overuse animations.

Type-Based Colors

Give Pokémon types visually distinct colors.

For example:

Type	Suggested Color
Fire	Red / Orange
Water	Blue
Grass	Green

Electric	Yellow
Psychic	Pink
Ghost	Purple
Ice	Cyan
Dragon	Indigo
Dark	Gray
Fairy	Pink

You may create your own color system.

Technical Requirements

You may use:

- React
- Next.js
- Vue
- Angular
- Svelte

Preferred stack:

React + TypeScript

You may use a CSS solution such as:

- CSS Modules
- Tailwind CSS
- Styled Components
- Plain CSS

For icons, you may use a library such as Lucide or another appropriate icon library.

Code Quality

Your code should demonstrate good frontend practices.

Requirements:

- Reusable components
- Clear folder structure
- Meaningful variable/function names
- Avoid unnecessary duplication
- Proper API error handling
- Proper loading states
- Responsive CSS
- Clean component architecture
- TypeScript types where applicable

Suggested structure:

```
src/
├── components/
│   ├── PokemonCard.tsx
│   ├── PokemonGrid.tsx
│   ├── SearchBar.tsx
│   ├── TypeFilter.tsx
│   ├── PokemonModal.tsx
│   ├── LoadingSkeleton.tsx
│   └── ErrorState.tsx
│
├── services/
│   └── pokemonApi.ts
│
├── types/
│   └── pokemon.ts
│
├── pages/
│
├── hooks/
│
└── styles/
```

You do not have to follow this structure exactly.

Bonus Features

These features are optional but can earn additional credit.

⭐ Favorites

Allow users to favorite Pokémon.

Persist favorites using:

localStorage

⭐ Dark Mode

Add:

☀️ Light
🌙 Dark

⭐ Sort

Allow sorting by:

- ID
- Name
- Attack
- Speed
- HP

⭐ Compare Pokémon

Allow users to select two Pokémon and compare their statistics.

Example:

	Pikachu	Charizard
HP	35	78
Attack	55	84
Defense	40	78
Speed	90	100

⭐ Keyboard Accessibility

Make the application usable with:

- Keyboard navigation
- Enter
- Escape
- Tab

⭐ URL-Based Search

Make the selected Pokémon shareable through the URL.

Example:

/pokemon/pikachu

Deliverables

Submit:

1. GitHub repository
2. Live deployed application
3. README.md
4. Screenshots of the application

The README should contain:

Pokémon Explorer

Features

Tech Stack

API Used

Installation

Running Locally

Project Structure

Challenges Faced

Future Improvements

Evaluation Criteria

Category	Weight
UI/Visual Design	25%
API Integration	20%
Functionality	20%
Responsive Design	15%
Code Quality	10%
Loading/Error/Empty States	5%
Extra Features	5%

Total: 100%

Final Challenge

The application should feel like a **real-world product**, not simply a page that displays API data.

A good submission should answer:

> "If this were released as a real website, would I enjoy using it?"

Focus on the complete experience:

API → Data → Components → UI → Interactions → Responsive Design → Error Handling

Make it visually impressive, intuitive, and production-quality.